# Extraction et identification du SBOM d'applications Android compilées

Les bibliothèques tierces peuvent introduire des vulnérabilités connues, répertoriées sous forme de **CVE (Common Vulnerabilities and Exposures)**. Pour identifier ces vulnérabilités, il est nécessaire de disposer du **SBOM (Software Bill of Materials)** de l'application, c'est-à-dire la liste exhaustive des dépendances logicielles et de leurs versions.

Dans le cas des **applications Android**, cette tâche est particulièrement complexe :

- les bibliothèques sont intégrées directement dans le **bytecode** de l'application (fichiers `.dex`) lors de la compilation,
- les informations de version ou de provenance ne sont pas toujours disponibles,
- les techniques d'**obfuscation** compliquent encore davantage l'analyse statique du code.

Malgré ces difficultés, plusieurs approches d'analyse de code et d'extraction de SBOM ont été proposées dans la littérature scientifique. Ce projet vise à explorer ces approches, à les évaluer, et à proposer une méthode efficace d'identification des composants logiciels d'une application Android compilée et potentiellement obfusquée dans l'optique de détecter les CVE associées.

## Objectifs du projet

1. **Étudier l'état de l'art** des techniques d'extraction de SBOM sur les applications Android compilées (APK) et potentiellement obfusquées.
2. **Expérimenter différentes approches** (analyse statique, décompilation, empreintes binaires, similarité de code, etc.) **et outils existants**
3. **Proposer et implémenter un prototype** permettant de :
   - extraire les bibliothèques présentes dans une APK,
   - estimer leurs versions
   - produire un SBOM
   - utiliser le SBOM pour identifier les CVE connues via des outils existants (syft, trivy) ou des bases de données publiques (NVD, OSS Index, etc.).
4. **Évaluer la précision** et la robustesse de la méthode proposée sur un ensemble d'applications Android dont les sources, et donc les versions de dépendances utilisées, sont disponibles. Utilisation des datasets des outils existants.

## Verrous

- **Obfuscation / transformation du code**. Les outils d'obfuscation (renommage, remaniement, fusion/fission de fonctions, contrôle de flux, inlining, etc.) rendent difficile l'identification des librairies et des signatures de code.
- **Absence ou perte de métadonnées versionnelles**. Dans le code compilé, les versions (numéros, signatures, noms des packages) peuvent ne plus apparaître.
- **Évolution / versioning**. Chaque librairie est associée à un certain nombre de versions. Une CVE concerne généralement une version ou un intervalle de version. Il faut donc être en mesure d'identifier chacune des versions d'une même librairie.

## Points d'entrées de l'état de l'art


- Zhang, Z., Luo, S., Lu, Y., & Pan, L. (2025). Obfuscation-resilient detection of Android third-party libraries using multi-scale code dependency fusion. *Information Fusion*, *117*, 102908.
- Zhou, B., Wu, J., Ling, X., Luo, T., & Zhang, J. (2025). Version-level Third-Party Library Detection in Android Applications via Class Structural Similarity.
- Pan, D., Xu, Y., Feng, R., Yu, D., Chen, J., Fang, Y., & Zhang, Y. (2025, March). Pay Your Attention on Lib! Android Third-Party Library Detection via Feature Language Model. In *2025 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)* (pp. 217-228). IEEE.
- Zhan, X., Fan, L., Chen, S., We, F., Liu, T., Luo, X., & Liu, Y. (2021, May). Atvhunter: Reliable version detection of third-party libraries for vulnerability identification in android applications. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)* (pp. 1695-1707)
- Zhan, X., Fan, L., Liu, T., Chen, S., Li, L., Wang, H., ... & Liu, Y. (2020, December). Automated third-party library detection for android applications: Are we there yet?. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering* (pp. 919-930).
- Backes, M., Bugiel, S., & Derr, E. (2016, October). Reliable third-party library detection in android and its security applications. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security* (pp. 356-367).
- Li, M., Wang, W., Wang, P., Wang, S., Wu, D., Liu, J., ... & Huo, W. (2017, May). Libd: Scalable and precise third-party library detection in android markets. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)* (pp. 335-346). IEEE.
- Wang, Y., Wu, H., Zhang, H., & Rountev, A. (2018, May). Orlis: Obfuscation-resilient library detection for android. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems* (pp. 13-23).
- Zhan, X., Liu, T., Liu, Y., Liu, Y., Li, L., Wang, H., & Luo, X. (2021). A systematic assessment on android third-party library detection tools. *IEEE Transactions on Software Engineering*, *48*(11), 4249-4273.

- Ramirez, P. M. (2024). Empirical Study of Third-Party Library Detection in iOS and Android Applications.